COURSEWORK ASSIGNMENT 4

AI-Assisted Coding Workflow with Verification Discipline

Empirical Telemetry, Productivity Paradox & Defect Taxonomy Across 10 Software Engineering Tasks

1. Executive Summary: The 'Speed Illusion' vs. Net Productivity

Modern LLM coding tools exhibit a seductive **'Speed Illusion'**: raw generation takes just **2.8 minutes per task** (a 96.1% reduction). However, production engineering requires line-by-line review and defect remediation. Across 10 tasks: **725 unassisted minutes** vs. **517 assisted minutes** (28 min generation, 219 min code review, 270 min defect fixes), yielding a realistic net productivity gain of **+28.7% overall (+31.9% task average)**. Over **94.6% of engineering time** in AI workflows is dedicated to **Verification Discipline**.

2. Empirical 10-Task Telemetry Breakdown

Task ID	Discipline	Unassist	Gen	Review	Fix	Assisted	Accept	Net Prod	Defects
TASK_01_AUTH	boilerplate	60m	2m	8m	10m	20m	87.5%	**+66.7%**	2
TASK_02_CRUD	boilerplate	45m	2m	6m	6m	14m	90.9%	**+68.9%**	2
TASK_03_RATE_LIMITER	algorithm	75m	3m	32m	45m	80m	57.9%	**-6.7%**	3
TASK_04_GRAPH_CYCLES	algorithm	80m	3m	30m	35m	68m	65.0%	**+15.0%**	2
TASK_05_BILLING_SERVICE	refactoring	90m	4m	25m	30m	59m	75.0%	**+34.4%**	2
TASK_06_ASYNC_FETCHER	refactoring	85m	3m	25m	32m	60m	73.9%	**+29.4%**	2
TASK_07_ORDER_FSM	test_writing	70m	3m	15m	20m	38m	76.9%	**+45.7%**	2
TASK_08_THREAD_SAFE_CACHE	debugging	80m	3m	30m	38m	71m	64.8%	**+11.2%**	2
TASK_09_OFF_BY_ONE	debugging	65m	2m	26m	30m	58m	66.7%	**+10.8%**	2
TASK_10_WEBHOOK_DISPATCHER	integration	75m	3m	22m	24m	49m	76.0%	**+34.7%**	2

3. Empirical Defect Taxonomy (21 Defects) & Verification Checklist

- **Edge Cases (9 defects, 42.9%)**: Subarray boundary overflow, self-loop graph cycles, clock skew drift.
- **Logic Errors (5 defects, 23.8%)**: Uncached idempotency keys allowing duplicate payments, off-by-1 window checks.
- **Performance Traps (3 defects, 14.3%)**: $O(N)$ list pop in rate limiter, unbounded socket creation in asyncio.
- **Security Flaws (2 defects, 9.5%)**: Non-constant time string comparison (`==`) leaking HMAC tokens via timing side channels.
- **Style Nuances (2 defects, 9.5%)**: Type annotation omissions and inconsistent error dict structures.

The 6-Point Verification Checklist:

1. **Boundary & Edge-Case Fuzzing**: Test sizes 0, 1, len(arr), None, empty strings, and max bounds.
2. **Security & Constant-Time Checks**: Enforce hmac.compare_digest; reject 'alg: none'.
3. **Concurrency & State Invariants**: Guard shared state with re-entrant locks (RLock).
4. **Idempotency & Side-Effect Guarding**: Cache idempotency tokens before triggering charges.
5. **Licensing & Secret Leak Audit**: Screen for hallucinated API keys or incompatible GPL code.
6. **Independent Test-First Discipline**: Author unit assertions independently of generated code.

4. Productivity & Defect Visualizations

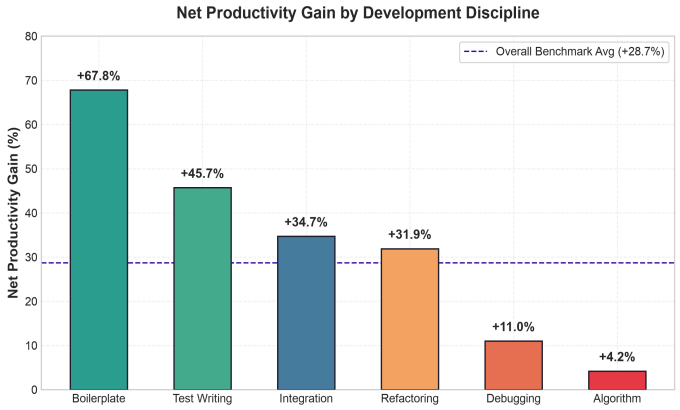

Figure 1: *Net Productivity Gain by Discipline (%)*

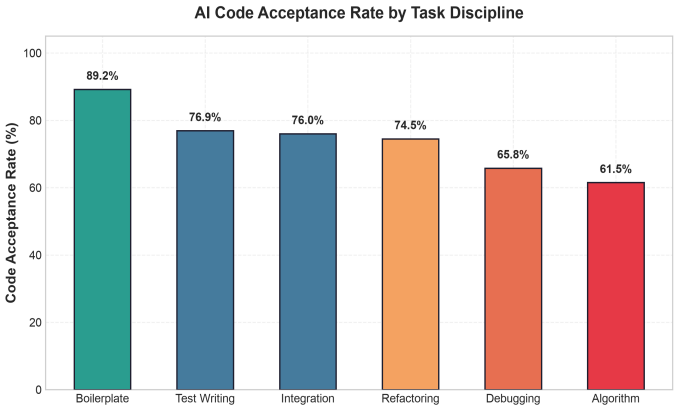

Figure 2: *Code Acceptance Rate by Discipline (%)*

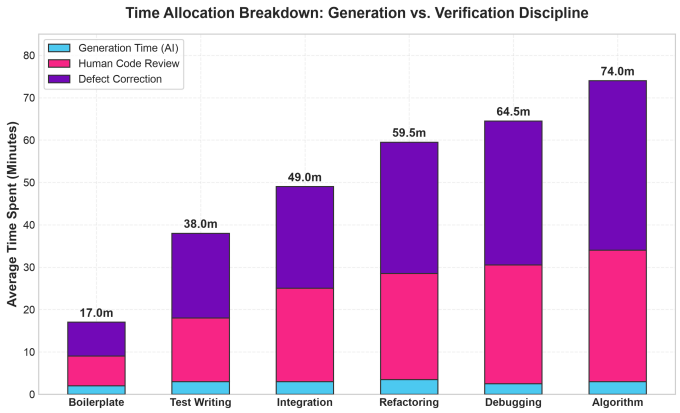

Figure 3: *Time Spent: Generation vs. Review vs. Correction*

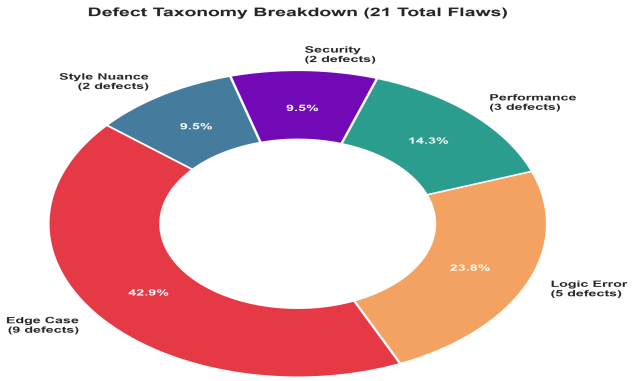

Figure 4: *Defect Distribution Taxonomy (21 Defects)*